

# **OPERATING SYSTEMS AND SYSTEM PROGRAMMING**

## **SKILL – WEEK 2**

Name: S Svara Haasini

Roll No: 2520090170

### **Task 1: Command History and Recall**

#### **Question:**

Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

## Command History Source Code

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ cat > history_demo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct HistoryNode {
    char *command;
    struct HistoryNode *prev;
    struct HistoryNode *next;
} HistoryNode;

HistoryNode *head = NULL;
HistoryNode *tail = NULL;
HistoryNode *current = NULL;

void add_history(const char *cmd)
{
    HistoryNode *node = malloc(sizeof(HistoryNode));

    if (node == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    node->command = malloc(strlen(cmd) + 1);

    if (node->command == NULL) {
        perror("malloc");
        free(node);
        exit(EXIT_FAILURE);
    }

    strcpy(node->command, cmd);

    node->prev = tail;
    node->next = NULL;

    if (tail != NULL)
        tail->next = node;
    else
        head = node;
    tail = node;

    EOF return 0;
}

// history memory released successfully.\n");;
```

## History Recall Execution

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ gcc -Wall -Wextra history_demo.c -o history_demo
haasini@SSH:~/OSSP_upload/Skill_Week2$ ./history_demo
=== Command History Demo ===
Commands: history, prev, next, exit

history_shell> ls
Stored: ls
history_shell> pwd
Stored: pwd
history_shell> echo hello
Stored: echo hello
history_shell> history

Command History:
1. ls
2. pwd
3. echo hello
history_shell> prev
prev
next
exitPrevious command: echo hello
history_shell> Previous command: pwd
history_shell> Next command: echo hello
history_shell> valgrind --version
Stored: exitvalgrind --version
history_shell> valgrind --leak-check=full ./history_demo
Stored: valgrind --leak-check=full ./history_demo
history_shell> ls
pwd
history
eStored: ls
history_shell> Stored: pwd
history_shell>
Command History:
1. ls
2. pwd
3. echo hello
4. exitvalgrind --version
5. valgrind --leak-check=full ./history_demo
6. ls
7. pwd
```

```

7. pwd
history_shell> ls
Stored: ls
history_shell> pwd
Stored: pwd
history_shell> history

Command History:
1. ls
2. pwd
3. echo hello
4. exitvalgrind --version
5. valgrind --leak-check=full ./history_demo
6. ls
7. pwd
8. ls
9. pwd
history_shell> exit
History memory released successfully.
haasini@SSH:~/OSSP_upload/Skill_Week2$ cat > parser_demo.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 256
#define MAX_TOKENS 20
#define TOKEN_SIZE 64

int tokenize(char *input, char tokens[][TOKEN_SIZE])
{
    int count = 0;
    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS) {
        strncpy(tokens[count], token, TOKEN_SIZE - 1);
        tokens[count][TOKEN_SIZE - 1] = '\\0';

        count++;
        token = strtok(NULL, " \\t\\n");
    }
}

```

### Observations:

- Commands entered by the user (ls, pwd, echo hello) are successfully stored in the command history.
- The history command displays all stored commands in numbered format (1-9 entries visible), maintaining complete historical record throughout the session.
- The prev command navigates backward through history, displaying 'Previous command: echo hello' then 'pwd', demonstrating reverse traversal of the linked-list history structure.
- The next command navigates forward through history, showing 'Next command: echo hello' after moving backward, confirming bidirectional navigation capability.
- The input buffer is updated and the currently selected history entry is maintained as the user navigates forward and backward through previous commands.
- Multiple commands including compound arguments (e.g., 'valgrind --leak-check=full

./history\_demo') are correctly stored and recalled, demonstrating support for complex input strings.

## Task 2: Dynamic Buffers and Memory Management

### Question:

Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

### Dynamic Memory Management

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ cat > history_demo.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct HistoryNode {
    char *command;
    struct HistoryNode *prev;
    struct HistoryNode *next;
} HistoryNode;

HistoryNode *head = NULL;
HistoryNode *tail = NULL;
HistoryNode *current = NULL;

void add_history(const char *cmd)
{
    HistoryNode *node = malloc(sizeof(HistoryNode));

    if (node == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    node->command = malloc(strlen(cmd) + 1);

    if (node->command == NULL) {
        perror("malloc");
        free(node);
        exit(EXIT_FAILURE);
    }

    strcpy(node->command, cmd);

    node->prev = tail;
    node->next = NULL;

    if (tail != NULL)
        tail->next = node;
    else
        head = node;
    tail = node;

    EOF return 0; history memory released successfully.\n");;
```

## Valgrind Memory Verification

```
history_shell> history

Command History:
1. ls
2. pwd
3. exitls
4. pwd
history_shell> exit
History memory released successfully.
==1797==
==1797== HEAP SUMMARY:
==1797==    in use at exit: 0 bytes in 0 blocks
==1797==   total heap usage: 11 allocs, 11 frees, 2,282 bytes allocated
==1797==
==1797== All heap blocks were freed -- no leaks are possible
==1797==
==1797== For lists of detected and suppressed errors, rerun with: -s
==1797== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
haasini@SSH:~/OSSP_upload/Skill_Week2$ |
```

### Observations:

- History nodes are dynamically allocated using `malloc(sizeof(HistoryNode))`, creating independent memory blocks for each stored command.
- Command strings are dynamically allocated with `malloc(strlen(cmd) + 1)`, allowing arbitrary-length command storage without fixed-size buffer constraints.
- The input buffer is managed using `getline()`, which dynamically allocates and resizes the input buffer as required, reducing the need for a fixed-size command-input buffer.
- Linked nodes maintain `prev` and `next` pointers, forming a doubly linked list that enables traversal through the history in both directions without buffer overflow risk.
- Allocated memory is explicitly released: `free_history()` traverses the linked list and releases each stored command string and node, while the input buffer allocated through `getline()` is released using `free(input)`.
- Valgrind verification was performed using `'valgrind --leak-check=full ./history_demo'`. The HEAP SUMMARY reports 0 bytes in 0 blocks in use at exit, with a total heap usage of 11 allocs, 11 frees, and 2,282 bytes allocated.
- Valgrind confirms 'All heap blocks were freed -- no leaks are possible', and the ERROR SUMMARY reports 0 errors from 0 contexts (suppressed: 0 from 0), verifying that all dynamically allocated memory in `history_demo` was correctly released.

### Task 3: Tokenization

#### Question:

Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

#### Tokenizer Source Code

```
7. pwd
history_shell> ls
Stored: ls
history_shell> pwd
Stored: pwd
history_shell> history

Command History:
1. ls
2. pwd
3. echo hello
4. exitvalgrind --version
5. valgrind --leak-check=full ./history_demo
6. ls
7. pwd
8. ls
9. pwd
history_shell> exit
History memory released successfully.
haasini@SSH:~/OSSP_upload/Skill_Week2$ cat > parser_demo.c <<'EOF'
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 256
#define MAX_TOKENS 20
#define TOKEN_SIZE 64

int tokenize(char *input, char tokens[][TOKEN_SIZE])
{
    int count = 0;
    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS) {
        strncpy(tokens[count], token, TOKEN_SIZE - 1);
        tokens[count][TOKEN_SIZE - 1] = '\\0';

        count++;
        token = strtok(NULL, " \\t\\n");
    }
}
```



## Tokenization Output

```
        token = strtok(NULL, " \\t\\n");
    }

    return count;
}

void display_parse_structure(char tokens[][TOKEN_SIZE], int count)
{
    if (count == 0) {
        printf("Parser result: EMPTY COMMAND\\n");
        return;
    }

    printf("\\nParse Structure\\n");
    printf("COMMAND\\n");
    printf("  |-- Program : %s\\n", tokens[0]);

    for (int i = 1; i < count; i++) {
        printf("  |-- Argument %d : %s\\n", i, tokens[i]);
    }
}

EOF return 0;arser terminated successfully.\\n");d successfully.\\n");
aasini@SSH:~/OSSP_upload/Skill_Week2$ gcc -Wall -Wextra parser_demo.c -o parser_demo
aasini@SSH:~/OSSP_upload/Skill_Week2$ ./parser_demo
=== Tokenizer and Parser Demo ===
Type 'exit' to stop.

parser> ls -l /home
Token count: 3
Token[0] = "ls"
Token[1] = "-l"
Token[2] = "/home"

Parse Structure
COMMAND
  |-- Program : ls
  |-- Argument 1 : -l
  |-- Argument 2 : /home

Execution structure created successfully.
```

Execution structure created successfully.

```
parser> gcc main.c -o main
```

Token count: 4

Token[0] = "gcc"

Token[1] = "main.c"

Token[2] = "-o"

Token[3] = "main"

Parse Structure

COMMAND

|-- Program : gcc

|-- Argument 1 : main.c

|-- Argument 2 : -o

|-- Argument 3 : main

Execution structure created successfully.

```
parser> echo hello world
```

Token count: 3

Token[0] = "echo"

Token[1] = "hello"

Token[2] = "world"

Parse Structure

COMMAND

|-- Program : echo

|-- Argument 1 : hello

|-- Argument 2 : world

Execution structure created successfully.

```
parser> exit
```

Parser terminated successfully.

haasini@SSH:~/OSSP\_upload/Skill\_Week2\$ ./parser\_demo

=== Tokenizer and Parser Demo ===

Type 'exit' to stop.

```
parser>
```

Parser result: EMPTY COMMAND

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ ./parser_demo
=== Tokenizer and Parser Demo ===
Type 'exit' to stop.

parser>
Parser result: EMPTY COMMAND

parser> cat parser_demo.c
Token count: 2
Token[0] = "cat"
Token[1] = "parser_demo.c"

Parse Structure
COMMAND
|-- Program : cat
|-- Argument 1 : parser_demo.c

Execution structure created successfully.

parser> exit
Parser terminated successfully.
```

### Observations:

- Input is split into individual tokens using `strtok(input, " \t\n")`, recognizing space, tab, and newline as delimiters.
- The `tokenize()` function iterates through all delimiters and extracts each token, storing up to `MAX_TOKENS` 20 tokens in a 2D array.
- Each token is stored with null-termination (`tokens[count][TOKEN_SIZE - 1] = '\0'`), ensuring safe string handling and preventing overflow.
- The first token represents the program/command name (e.g., 'ls', 'gcc', 'echo', 'cat'), while subsequent tokens represent arguments.
- Whitespace handling correctly separates multi-word commands: 'echo hello world' is tokenized into 3 separate tokens: 'echo', 'hello', 'world'.
- Complex command arguments with special characters (e.g., 'gcc main.c -o main', 'ls -l /home') are correctly tokenized into individual components.
- Token count is returned, allowing validation of the total number of tokens extracted from each input line.
- The debug output shows token validation: each token is printed with its index and value, confirming correct parsing.

## Task 4: Parser Logic and Execution Structure

### Question:

Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

### Parse Structure

```
        token = strtok(NULL, " \\t\\n");
    }

    return count;

void display_parse_structure(char tokens[][TOKEN_SIZE], int count)

{
    if (count == 0) {
        printf("Parser result: EMPTY COMMAND\\n");
        return;
    }

    printf("\\nParse Structure\\n");
    printf("COMMAND\\n");
    printf("  |-- Program : %s\\n", tokens[0]);

    for (int i = 1; i < count; i++) {
        printf("  |-- Argument %d : %s\\n", i, tokens[i]);
    }

}

EOF return 0;arser terminated successfully.\\n");d successfully.\\n");
naasini@SSH:~/OSSP_upload/Skill_Week2$ gcc -Wall -Wextra parser_demo.c -o parser_demo
naasini@SSH:~/OSSP_upload/Skill_Week2$ ./parser_demo
=== Tokenizer and Parser Demo ===
Type 'exit' to stop.

parser> ls -l /home
Token count: 3
Token[0] = "ls"
Token[1] = "-l"
Token[2] = "/home"

Parse Structure
COMMAND
  |-- Program : ls
  |-- Argument 1 : -l
  |-- Argument 2 : /home

Execution structure created successfully.
```

```
Execution structure created successfully.
```

```
parser> gcc main.c -o main
```

```
Token count: 4
```

```
Token[0] = "gcc"
```

```
Token[1] = "main.c"
```

```
Token[2] = "-o"
```

```
Token[3] = "main"
```

```
Parse Structure
```

```
COMMAND
```

```
|-- Program : gcc
```

```
|-- Argument 1 : main.c
```

```
|-- Argument 2 : -o
```

```
|-- Argument 3 : main
```

```
Execution structure created successfully.
```

```
parser> echo hello world
```

```
Token count: 3
```

```
Token[0] = "echo"
```

```
Token[1] = "hello"
```

```
Token[2] = "world"
```

```
Parse Structure
```

```
COMMAND
```

```
|-- Program : echo
```

```
|-- Argument 1 : hello
```

```
|-- Argument 2 : world
```

```
Execution structure created successfully.
```

```
parser> exit
```

```
Parser terminated successfully.
```

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ ./parser_demo
```

```
=== Tokenizer and Parser Demo ===
```

```
Type 'exit' to stop.
```

```
parser>
```

```
Parser result: EMPTY COMMAND
```

## Empty Command Handling

```
haasini@SSH:~/OSSP_upload/Skill_Week2$ ./parser_demo
=== Tokenizer and Parser Demo ===
Type 'exit' to stop.

parser>
Parser result: EMPTY COMMAND

parser> cat parser_demo.c
Token count: 2
Token[0] = "cat"
Token[1] = "parser_demo.c"

Parse Structure
COMMAND
|-- Program : cat
|-- Argument 1 : parser_demo.c

Execution structure created successfully.

parser> exit
Parser terminated successfully.
```

### Observations:

- The parser receives tokens from the tokenizer and constructs a simplified parse structure representing the command hierarchy.
- The first token (tokens[0]) is identified as the program/executable name and displayed as 'Program' in the parse structure.
- Remaining tokens (tokens[1] through tokens[count-1]) are displayed as 'Argument 1', 'Argument 2', etc., establishing the argument list.
- Empty commands (count == 0) are detected and reported as 'Parser result: EMPTY COMMAND', preventing execution of null inputs.

- The simplified parse structure uses a hierarchical text representation:

COMMAND

└─ Program : [program\_name]

└─ Argument 1 : [arg1]

└─ Argument 2 : [arg2]

This structure is produced successfully for each valid command, as confirmed by the message 'Execution structure created successfully.'

- The parser handles various command formats: simple commands (cat parser\_demo.c), complex commands with options (gcc main.c -o main), and commands with multiple arguments (echo hello world).
- The structured command representation is suitable for later execution by providing clear separation between the program and its arguments.